KamiLimu ICT Track Industry Case Study

Vendor Payment & Approval Management System

Background

ImaraWorks Ltd. is a fictional medium-sized Kenyan construction company managing several projects across the country.

The company works with many external parties, including:

- building-material suppliers;
- transport and logistics providers;
- subcontractors;
- equipment-rental companies;
- consultants; and
- casual service providers.

As the company has grown, the number of payments it processes every week has increased significantly.

However, its payment process has not changed.

Today, an employee who needs a supplier paid usually sends a message to the finance team or a manager through WhatsApp.

A typical request might look like:

> Please pay KES 84,500 to ABC Hardware for electrical materials delivered to the Westlands project. Invoice INV-2043.

Finance then confirms the request, asks for approval where necessary, and eventually makes the payment through M-Pesa or a bank transfer.

Information about these requests is spread across WhatsApp conversations, spreadsheets, invoices, emails and payment confirmations.

This process worked when the organisation was smaller, but it is becoming difficult to manage.

The Problem

ImaraWorks has identified several challenges with its current payment process.

1. Payment requests can get lost

Requests arrive through different channels, making it difficult for employees to know whether their request has been received, approved or paid.

2. Managers have become a bottleneck

Many payments require senior employees to manually check messages and approve requests.

This becomes difficult when managers are travelling, in meetings or unavailable.

3. There is limited accountability

When reviewing an old transaction, it can be difficult to determine:

- who requested the payment;
- who approved it;
- when it was approved;
- whether any information changed;
- who processed the payment; and
- why a request was rejected.

4. Duplicate payments are possible

The same invoice may accidentally be submitted twice.

A payment may also be retried if a bank or M-Pesa transaction appears to fail, even though the first transaction may actually have succeeded.

5. Reconciliation is largely manual

At the end of the week or month, the finance team must compare payment requests against actual transactions.

6. Employees have poor visibility

Employees frequently contact finance simply to ask:

> Has my supplier been paid?

The company believes that software could make this process more efficient, transparent and reliable.

Your Challenge

You have been hired as the software engineering team responsible for proposing and developing a solution.

Your task is to:

> **Design and build a system that allows ImaraWorks to manage vendor payment requests from submission through approval and payment tracking.**

The system should reduce manual coordination while ensuring that payments remain properly authorised and traceable.

You are responsible for deciding how the system should be designed.

There may be several valid solutions.

Users of the System

At minimum, consider the following users.

Employee / Requester

An employee can submit a payment request and track its progress.

Manager

A manager can review requests requiring their approval.

Finance Officer

Finance can review approved requests, initiate payment processing and reconcile completed transactions.

Administrator

An administrator can manage users and access information required for system administration.

You may introduce additional roles if your design requires them.

Core Requirements

1. Payment Requests

An employee should be able to create a payment request.

A request should contain enough information for someone else to understand and process it.

For example:

- vendor;
- amount;
- reason for payment;
- invoice/reference number;
- project or department;
- requested payment method;
- date;
- requester; and
- supporting notes.

You must decide which fields are required and how they should be validated.

---

2. Request Status

Users should be able to determine the current state of a payment request.

Possible states may include:

```
Draft
Submitted
Pending Approval
Approved
Rejected
Processing
Paid
Failed
```

You may modify this workflow if you can justify your design.

Think carefully about:

- which transitions should be allowed;
- who can perform each transition;

- what information must be captured during a transition; and
- whether an action can be reversed.

---

3. Approval

Payment requests should not automatically be paid simply because an employee submitted them.

The system should support an approval process.

For the initial version:

- payments of **KES 50,000 or less** require approval from one manager;
- payments **above KES 50,000** require approval from a manager and finance;
- a user must not approve a payment request they created themselves; and
- rejected requests must include a reason.

Think about what should happen if the payment amount changes after approval.

---

4. Payment Processing

For this case study, you are **not required to integrate with a real bank or M-Pesa API**.

Instead, build or simulate a payment provider.

For example:

```
Payment Request
       ↓
    Approval
       ↓
Payment Service
       ↓
 Mock Provider
       ↓
Success / Failure
```

The mock provider may simulate responses such as:

```
{
  "reference": "PAY-89231",
```

```
  "status": "SUCCESS"
}
```

or:

```
{
  "reference": "PAY-89232",
  "status": "FAILED",
  "reason": "Provider unavailable"
}
```

Your application should react appropriately to these responses.

---

5. Duplicate Payments

The company is particularly concerned about paying the same invoice twice.

Your solution should provide reasonable safeguards against duplicate requests or duplicate payments.

Consider scenarios such as:

- the same invoice being submitted twice;
- an employee clicking "Pay" more than once;
- a network request being repeated;
- payment processing timing out.

You should explain the approach your team chooses.

---

6. Audit History

Important actions should be traceable.

For a payment request, the company should be able to determine things such as:

```
10:03 — Lewis created request
10:06 — Lewis submitted request
10:31 — Manager approved request
10:42 — Finance approved request
10:45 — Payment processing started
10:46 — Payment completed
```

Your team should decide which actions must be recorded.

---

7. Search and Reporting

Finance should be able to find and review payment requests.

Consider supporting filters such as:

- vendor;
- status;
- requester;
- project;
- date range; and
- payment method.

Management should also have access to some basic information about payment activity.

For example:

- total amount paid;
- requests awaiting approval;
- failed payments;
- payments grouped by project; or
- payments made during a particular period.

You do not need to build an advanced analytics platform.

Sample Information

You may use the following fictional data when developing your application.

Users

Name	Role
Alice Mwangi	Employee
Brian Otieno	Employee
Carol Wanjiku	Manager
David Mutua	Manager
Faith Njeri	Finance Officer
Grace Kamau	Administrator

Vendors

Vendor	Service
Metro Hardware Ltd.	Construction materials
Prime Cement Supplies	Cement
SwiftHaul Logistics	Transport
PowerHire Kenya	Equipment rental
Apex Electricals	Electrical materials
BlueLine Plumbing	Plumbing services

Example Requests

Vendor	Amount	Invoice	Purpose
Metro Hardware Ltd.	KES 24,500	INV-1032	Site materials
SwiftHaul Logistics	KES 67,000	SH-820	Material transport
Apex Electricals	KES 148,000	AE-4103	Electrical installation
PowerHire Kenya	KES 42,000	PH-912	Excavator rental

You may create additional test data.

Technical Constraints

Your team should build a working software solution.

The solution should:

- store information in a persistent database;
- have a clear separation of responsibilities within the codebase;
- implement authentication or an equivalent user identification mechanism;
- enforce permissions according to user roles;
- validate important inputs;
- contain automated tests for important business rules;
- use Git and GitHub for version control;
- demonstrate collaborative development through branches, commits and code reviews;
- include documentation explaining how the application works; and
- be deployable so that reviewers can access or run the solution.

You may use the programming language, framework and database agreed upon during the ICT track.

A beautiful interface is welcome, but **correct behaviour and good engineering decisions are more important than visual design**.

Responsible Computing

Payments contain sensitive organisational and personal information.

Your team must consider responsible computing when designing the solution.

Think about questions such as:

- Should every employee see every payment?
- Who should see financial information?
- What information should administrators be able to access?
- Should payment records be permanently deleted?
- How should sensitive information appear in logs?
- How could someone abuse the approval system?
- What should happen when an employee leaves the company?
- How should the system prevent unauthorised actions?

Your final presentation should explain at least **three responsible-computing decisions** made by your team.

Use of AI

You may use AI tools to support your software engineering work.

However, your team remains responsible for understanding and validating the software you submit.

Your README should briefly state:

- which AI tools you used;
- what you used them for;
- one example where AI helped your team; and
- one example where your team had to verify, modify or reject AI-generated work.

You should be able to explain any code contained in your final solution.

Out of Scope

You do **not** need to implement:

- a real M-Pesa integration;
- a real banking integration;
- QuickBooks or accounting integrations;
- actual movement of money;
- SMS services;
- email services;
- vendor KYC;
- payroll;
- tax calculation; or
- enterprise-grade infrastructure.

Mock these services where necessary.

Focus on the **core software engineering problem**.

Expected Deliverables

Each team should submit the following.

1. Working Application

A functional implementation demonstrating the core payment-request workflow.

2. GitHub Repository

The repository should contain:

- source code;
- meaningful commit history;
- README;
- instructions for running the application;
- tests; and
- any required configuration examples.

Do not commit passwords, API keys or other secrets.

3. Architecture Diagram

Create a simple diagram showing the major components of your system and how they communicate.

For example:

```
User
  ↓
Application
  ↓
Business Logic
  ↓
Database

     +

Mock Payment Provider
```

This example is intentionally simplified. Your team should document its own architecture.

4. Technical Documentation

Briefly explain:

- your architecture;
- important data models;
- major design decisions;
- assumptions;
- testing approach;
- security and responsible-computing considerations; and
- known limitations.

5. Deployed Solution

Where practical, deploy the application so that it can be demonstrated remotely.

6. Team Presentation

Prepare a short presentation covering:

1. your understanding of the problem;
2. your solution;
3. a demonstration;
4. architecture;
5. important engineering decisions;
6. testing;
7. responsible-computing considerations;
8. challenges encountered; and
9. what you would improve with more time.

What We Are Looking For

This case study is **not a competition to build the largest application**.

A smaller system that behaves correctly is better than a large system whose behaviour is difficult to understand or unreliable.

We will particularly be interested in:

- how you interpreted the problem;
- the decisions your team made;
- how you modelled the workflow;
- how well your code reflects your design;
- how you tested important behaviour;
- how your team collaborated;
- how you dealt with ambiguity; and
- how clearly you can explain your decisions.

There is no single correct implementation.

Your objective is to demonstrate how you think and work **as a software engineering team**.